

TLS 1.3 support in HSP

In August 2018, RFC 8446ⁱ was officially published with the specification of TLS 1.3 - making it the official standard for transport encryption. The new version represents a significant step forward: it is cleaner, faster, and more secure in comparison to its predecessor.

A big step forward

One significant difference concerning speed is the reduced latency when performing a full handshake. Whereas previous versions of the TLS protocol require two complete roundtrips, with TLS 1.3 only one round-trip is required for clients to be able to send application data. In addition to this, servers can already send application data in response to the client's first handshake message. This means network latency has less impact on the time required to establish a secure connection.

RSA key exchange is known to be problematic because of not being "forward-secret": if someone records encrypted conversations and then gets hold of the server's RSA private key at some point later in time, all conversations can be decrypted. Although many organizations are aware of this weakness and have moved to Diffie-Hellman key agreement, RSA key exchange is still in place. TLS 1.3 eliminates RSA key exchange and mandates Perfect Forward Secrecy. The same is true for Diffie-Hellman parameters which not all are known to be safe. Here too, TLS 1.3 changes tune by restricting the Diffie-Hellman parameters to those known to be secure.

In the past, several attacks on the symmetric encryption and integrity of TLS have been demonstrated, such as the Poodle (Padding Oracle on downgraded Legacy Encryptionⁱⁱ) attack. Such "padding-oracle" attacks exploit the "MAC-then-Encrypt" approach of SSL and TLS up to version 1.2, at which a check value is first formed over the plain text (the so-called Message Authentication Code or MAC), then appended to the data, and finally encrypted all together. The attack vector is to guess the plain text byte by byte and let the integrity assurance tell you whether the guess is right or not. In TLS 1.3 "MAC-then-Encrypt" has been replaced by Authenticated Encryption with Associated Data (AEAD), which performs integrity assurance and encryption in one step - a technique that is considered much more secure.

Let's move on to the other "victims" of the TLS 1.3 cleanup, such as the Cipher Block Chaining

(CBC) mode, which was responsible for BEASTⁱⁱⁱ and Lucky 13^{iv} attacks. It is prohibited in TLS 1.3. Same for all crypto procedures considered to be exploitable or even known to be broken. This affects, for example, DES/3DES, MD5, SHA-1, RC4, and the weak "export cipher suites". As a conclusion, downgrade attacks like FREAK^v and Logjam^{vi} are no longer possible with TLS 1.3.

Two other weakening options have also been removed from the standard: compression and renegotiation of an existing connection. In other words, everything that caused trouble in the past or has become outmoded and legacy has been dropped in TLS 1.3.

As a replacement for the insecure cryptographic methods, new secure methods based on elliptic curves (ECC) are available. In addition, signing is done as often as possible - including the entire handshake. For example, the list of encryption suites supported by the client can no longer be manipulated in the same way as they were exploited with man-in-the-middle attacks on earlier versions. By encrypting the certificates, TLS 1.3 makes man-in-the-middle attacks even more difficult - or almost impossible.

All in all, the new version is a big step forward in securing data and speeding up network communication. And of course, HSP, which largely relies on security features provided by TLS, benefits greatly from these innovations of the newly appointed standard.

Starting with version 4.21, HSP supports TLS 1.3 throughout. That is, both incoming connections from clients as well as outgoing connections to backend servers can be secured using TLS 1.3.

There are a few special use cases, where limitations are to be considered when switching to TLS 1.3. Please refer to the section "Limitations" for further details.

At the same time, the support for SSL 3.0 has been dropped as the protocol no longer meets the current security standards^{vii}.

How to enable TLS 1.3 in HSP 4.21 and above

Client-side

Towards the client, the usage of TLS 1.3 can be configured using the well-known *SSLProtocol*^{viii} directive in HTS. It is important to mention that the limitation of using *SSLProtocol* with multiple name-based virtual hosts has been resolved. Now when SNI is provided in the TLS handshake by the client, the TLS protocol of each name-based virtual host can be honored. For all details please see the official documentation of *SSLProtocol*.

When using TLS 1.3, only the following five cipher suites are allowed, whereas the first three are used by default:

- TLS_AES_256_GCM_SHA384
- TLS_CHACHA20_POLY1305_SHA256
- TLS_AES_128_GCM_SHA256
- TLS_AES_128_CCM_8_SHA256
- TLS_AES_128_CCM_SHA256

To configure TLS 1.3 ciphers *SSLCipherSuite*^{ix} can be used with the newly introduced modifier "TLSv1.3".

```
SSLCipherSuite ECDHE-ECDSA-CHACHA20-  
POLY1305:ECDHE-RSA-CHACHA20-POLY1305:...
```

```
SSLCipherSuite TLSv1.3  
TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128  
_GCM_SHA256
```

In the example above, the first *SSLCipherSuite* directive sets the ciphers to be used by TLS versions up to 1.2. The second setting defines the ones to be used by TLS 1.3.

Note: changing the cipher list of TLS 1.2 (and below) has no impact on TLS 1.3 cipher suite configuration.

Backend-side

Towards the backends, the directive *HGW_SslClientProtocol* can be used in SRM to enable TLS 1.3 for outbound connections. When not set, the default as set by *HGW_SslClientProtocolDefault* (default is TLS 1.2) is used.

The directive *HGW_SslClientCipherSuite* has been extended to work the same way as *SSLCipherSuite* works for client-side connections. That is, the protocol specifier "TLSv1.3" can be used to set TLSv1.3 specific cipher-suites separately.

```
HGW_SslClientCipherSuite TLSv1.3  
TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128  
_GCM_SHA256
```

mod_proxy

When using functionalities provided by *mod_proxy* - like WebSockets or backend health checks - TLS 1.3 can be enabled using *SSLProxyProtocol* ^xdirective.

Please note that the directive *SSLProxyCipherSuite* works analogue to *SSLCipherSuite*.

```
<Proxy balancer://mybalancer>  
  SSLProxyProtocol TLSv1.3  
  SSLProxyCipherSuite TLSv1.3  
  TLS_CHACHA20_POLY1305_SHA256:TLS_AES_128  
  _GCM_SHA256  
  BalancerMember https://example.com:13245  
</Proxy>
```

Limitations

The fundamental changes introduced in TLS 1.3 also result in limitations compared to dedicated "use cases" with TLS 1.2 and before. Some of them are rather general, others are relevant to the use of TLS 1.3 within HSP only.

Post Handshake Authentication

As already mentioned at the beginning, TLS 1.3 does not support renegotiation. In earlier versions, renegotiation was - for example - often used to enable mutual/client authentication at a "virtual host wide" level. TLS 1.3 instead provides the "post handshake authentication" (PHA) extension, which allows a server to request a client's certificate at any time after the handshake.

However, this extension is optional and not required for the basic functionality of TLS 1.3 to work. In fact, most browsers do not yet support this extension (status as of April 2020), not least because of some uncertainties^{xi}.

It should be noted that the following two functionalities are based on PHA and therefore **cannot** be used with TLS 1.3 in conjunction with client software not supporting the PHA extension.

- Secure Handshake (SHS)
- *SSLRequire*^{xii} on location level (it does work however at virtual host level)

SSL ID locking

The documentation of OpenSSL 1.1.1 states the following regarding the option *SSL_OP_NO_TICKET*^{xiii}.

«The TLSv1.3 protocol only supports tickets and does not directly support session ids. However, OpenSSL allows two modes of ticket operation in TLSv1.3: stateful and stateless. Stateless tickets work the same way as in TLSv1.2 and below. Stateful tickets mimic the session ID behavior available in TLSv1.2 and below. The session information is cached on the server and the session

id is wrapped up in a ticket and sent back to the client. When the client wishes to resume, it presents a ticket in the same way as for stateless tickets. The server can then extract the session id from the ticket and retrieve the session information from its cache. »

In HSP the behavior can be controlled by the *SSLSessionTickets*^{xiv} directive, and if set to "off", the stateful mode of TLS 1.3 is enabled. For security reasons, OpenSSL 1.1.1 may hand out multiple stateful *SessionTickets* per connection so that clients are motivated to use each *Session-Ticket* only once (even though it is not strictly necessary). Investigations have shown that well-known browsers follow this best practice and use *SessionTickets* only once.

The consequence is that the TLS session ID changes on each request, which in turn leads to "false positives" when using HSP's *Session ID Lock* and *Advanced Session ID Lock Rules* features together with TLS 1.3.

In other words: When using TLS 1.3 for secure communication, HSP version 4.21 does not yet provide the ability to correlate session IDs and

thus guarantee the functionality of the mentioned features.

However, we work on providing a more sophisticated solution for this in future versions. In the meantime, customers must adapt *Session ID Lock Rules* accordingly or fall back to the usage of TLS 1.2. Please remind that with TLS 1.2, session tickets must be disabled in order to support *Session ID Lock* and *Advanced Session ID Lock Rules*.

Renegotiation issue with TLS 1.1/1.2

Tests with plain Apache httpd version 2.4.43 and OpenSSL 1.1.1 have revealed the existence of an issue with TLS renegotiation on backend connections: when backend servers initiate a TLS renegotiation, httpd's TLS proxy engine behaves incorrectly leading to a "handshake failure" and finally to the abortion of the connection. As HSP uses httpd's TLS proxy engine, the issue can also affect outgoing connections on SRM to backend servers. Note however that the issue has been reported^{xv} to Apache httpd and as soon as a solution is found, a new version of HSP will be made available.

ⁱ The Transport Layer Security (TLS) Protocol Version 1.3, <https://tools.ietf.org/html/rfc8446>

ⁱⁱ This POODLE Bites: Exploiting The SSL 3.0 Fallback, <https://www.openssl.org/~bodo/ssl-poodle.pdf>

ⁱⁱⁱ How the BEAST Attack Works, <https://www.netsparker.com/blog/web-security/how-the-beast-attack-works/>

^{iv} Lucky 13 Attack Explained, <https://medium.com/@c0D3M/lucky-13-attack-explained-dd9a9fd42fa6>

^v The FREAK Attack, <https://censys.io/blog/freak>

^{vi} Weak Diffie-Hellman and the Logjam Attack, <https://weakdh.org/>

^{vii} Deprecating Secure Sockets Layer Version 3.0, <https://tools.ietf.org/html/rfc7568>

^{viii} https://httpd.apache.org/docs/current/mod/mod_ssl.html#sslprotocol

^{ix} https://httpd.apache.org/docs/current/mod/mod_ssl.html#sslsuiter

^x https://httpd.apache.org/docs/current/mod/mod_ssl.html#sslproxyprotocol

^{xi} <https://bugs.chromium.org/p/chromium/issues/detail?id=911653>

^{xii} https://httpd.apache.org/docs/current/mod/mod_ssl.html#sslrequire

^{xiii} SSL_OP_NO_TICKET, https://www.openssl.org/docs/man1.1.1/man3/SSL_CTX_set_options.html

^{xiv} https://httpd.apache.org/docs/trunk/mod/mod_ssl.html#sslsessiontickets

^{xv} https://bz.apache.org/bugzilla/show_bug.cgi?id=64437